

1 – Introduction

Machine Learning (MPA-MLR)

Tomáš Vičar

2025

Department of Biomedical Engineering,
FEEC, Brno University of Technology

Course Outline

1. Introduction, optimization, overview
2. Model evaluation and pre-processing
3. Feature selection and linear models I
4. Linear models II, kernel methods
5. Bias–variance tradeoff, bagging, boosting, random forest
6. Neural networks
7. Deep learning I
8. Deep learning II
9. Deep learning III
10. Deep learning IV
11. Probabilistic models I
12. Probabilistic models II

The course evaluation rules will be addressed in the practical exercises.

Definitions

Branches of Machine Learning

Recapitulation of simplest algorithms (from BPC-UIN)

Optimization

Definitions

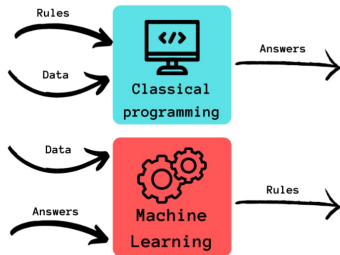
Classical Programming vs. Machine Learning

Classical Programming

- Human provides **rules** (algorithms) + **data**.
- Computer executes rules to produce **answers**.
- Example: Sorting numbers, calculating taxes, signal FIR filtering

Machine Learning

- Human provides **data** + **answers (labels)**.
- Computer learns the **rules** (model) from the data.
- Example: Spam filter, image classification, LLM (Large Language Model)



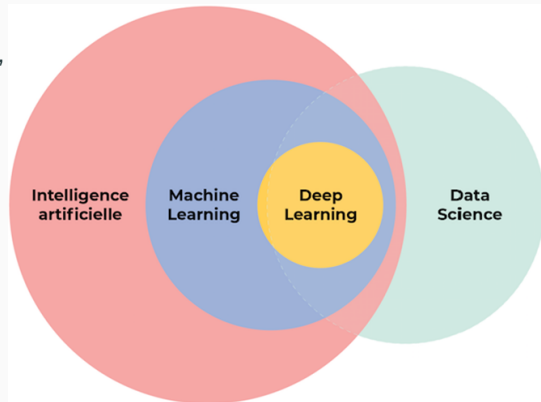
Artificial Intelligence vs. Machine Learning

Artificial Intelligence (AI)

- Broad field of computer science focused on creating systems that perform tasks requiring human intelligence.
- understanding language, decision-making, autonomous driving.
- **AI = umbrella term.**

Machine Learning (ML)

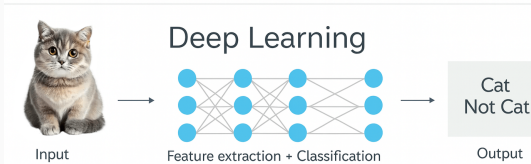
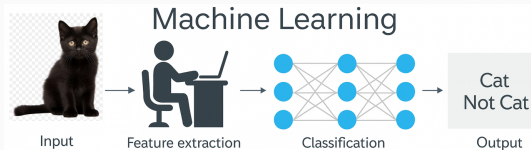
- Subset of AI focused on learning patterns from data.
- spam detection, recommender systems, medical image analysis
- **ML = one approach to achieve AI.**



Machine Learning vs. Deep Learning

- **Machine Learning:** Algorithms learn patterns from data, often need **manual feature engineering**, work well with smaller datasets.
- **Deep Learning:** Subset of ML using **neural networks with many layers**, learns features **automatically**, needs big data + high compute.

Machine Learning	Deep Learning
Manual features	Automatic features
Small data OK	Needs big data
Traditional algorithms	Neural networks



Related fields

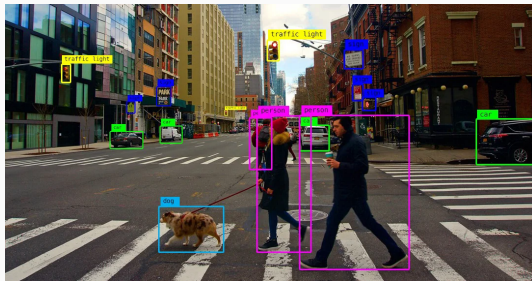
Data Science

- Interdisciplinary field that combines **statistics**, **machine learning**, and **computer science**.
- Goal: extract **insights and knowledge** from structured and unstructured data.
- Applications: business analytics, healthcare, recommender systems.



Computer Vision

- Subfield of Artificial Intelligence (AI).
- Enables computers to **derive meaningful information** from digital images, videos, and other visual inputs.
- Systems can **act or make recommendations** based on that information.
- Applications: object detection, medical imaging, autonomous driving.



Robotics

- Field focused on **designing, building, and programming robots**.
- Aims to enable robots to operate in **complex, real-world scenarios**.
- Combines AI, computer vision, control theory, and mechanics.
- Applications: industrial automation, service robots, autonomous vehicles.

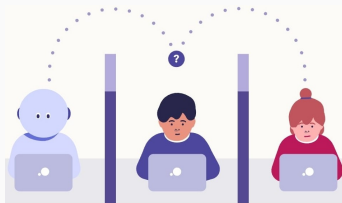


The Turing Test

AI raises the question: does **intelligent behavior** imply or require the existence of a **mind**? Can machines truly "think," or only **simulate** thinking?

The Turing Test (Alan Turing 1950)

- A test of a machine's ability to exhibit **intelligent behavior**.
- If a human evaluator cannot reliably distinguish between responses from a machine and a human, the machine is said to have demonstrated AI.
- LLMs **pass the Turing Test** - does this mean they are intelligent?



Narrow AI vs. General AI

Narrow AI (Weak AI)

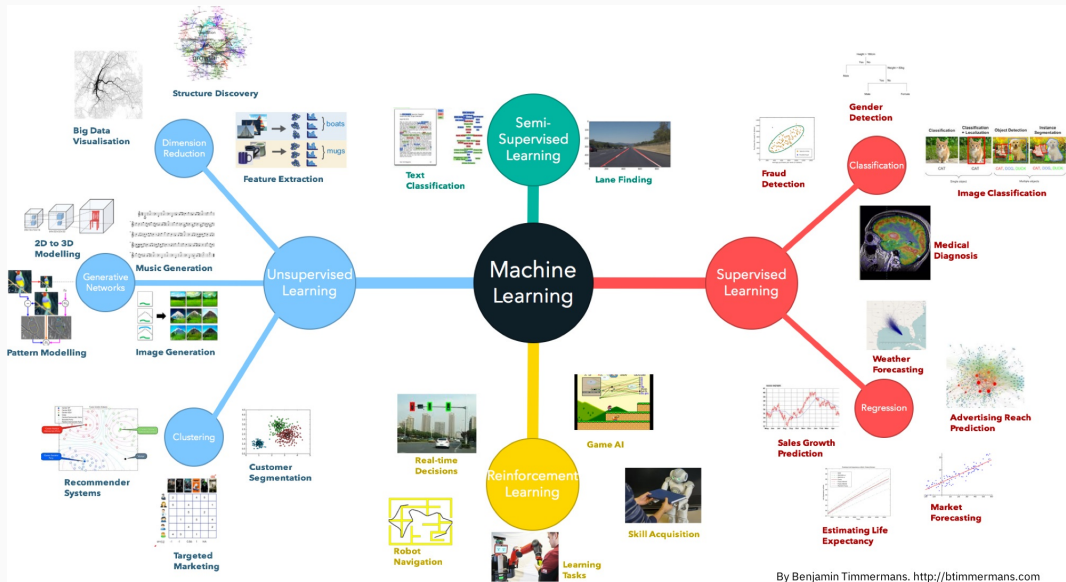
- AI systems designed for a **specific task**.
- Very good at what they are trained for, but cannot generalize.
- Examples: spam filters, recommendation systems, image classifiers, voice assistants.

General AI (Strong AI)

- Hypothetical AI with the ability to **understand, learn, and apply knowledge across domains**.
- Would perform any intellectual task that a human can do.
- Still a **theoretical concept**.
- AGI – Artificial General Intelligence

Branches of Machine Learning

Branches of Machine Learning



Supervised Learning

Definition

- A machine learning paradigm where models learn from **labeled data**.
- Training data consists of input variables (x) and known outputs (y).
- Goal: learn a mapping $f(x) \rightarrow y$ that generalizes to unseen data.
- Example: Cat vs. Dog recognition



Sample of cats & dogs images from Kaggle Dataset



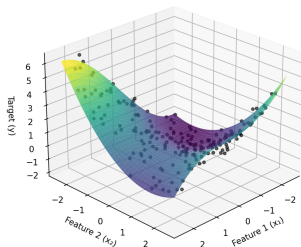
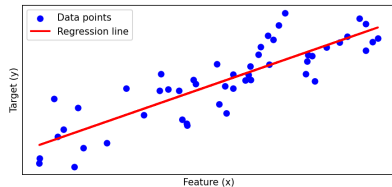
Regression

Definition

- Regression is a supervised learning technique used to model the relationship between input variables (x) (*features*) and a continuous output variable y (*target*).
- The goal is to predict or estimate the target value based on new input data.

Basic Concept

- Fits a function $\hat{y} = f(x)$ to minimize the error between predicted $\hat{y}$ and true values.
- Example: predicting house prices, predicting cholesterol level



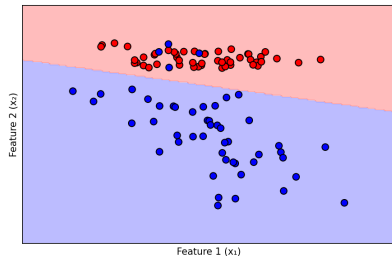
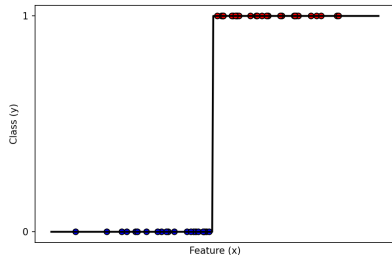
Classification

Definition

- Classification is a supervised learning technique used to assign input variables (x) (*features*) into one of several discrete categories (*classes*).
- The goal is to predict the class label y for new, unseen data.

Basic Concept

- Learns a decision function $\hat{y} = f(x)$ that separates data into distinct classes.
- Example: predicting **cat vs. dog**, classifying **healthy vs. diseased** patients.

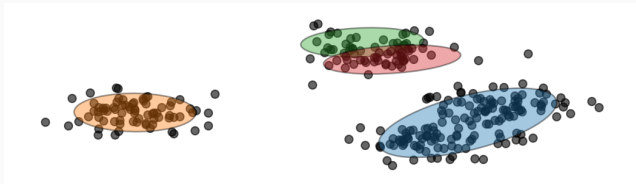


Definition

- A machine learning paradigm where models learn from **unlabeled data**.
- Only input variables (x) are given, without known outputs (y).
- Goal: discover **hidden patterns, structures, or groupings** in data.

Examples

- **Clustering:** grouping similar data (e.g. customers by shopping behavior)
- **Dimensionality Reduction:** compressing high-dimensional data (e.g., PCA for visualisation)
- **Generative Models:** learning distribution to create new samples (image generation)



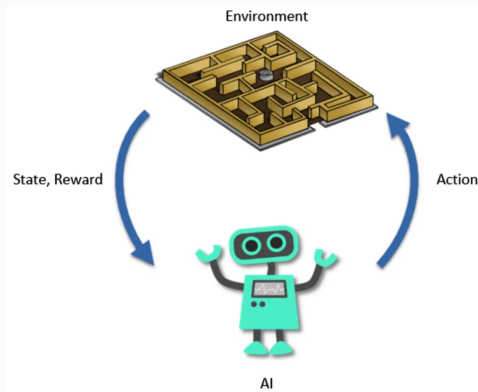
Reinforcement Learning

Definition

- A machine learning paradigm where an **agent** learns to make decisions by interacting with an **environment**.
- The agent observes the **state**, takes an **action**, and receives a **reward**.
- Goal: learn a policy that maximizes cumulative reward over time.

Examples

- Training robots to navigate mazes
- Game playing (e.g., AlphaGo, Atari games)
- Dynamic resource allocation



Self-Supervised Learning

Definition

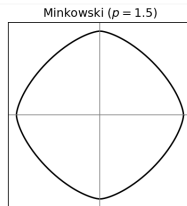
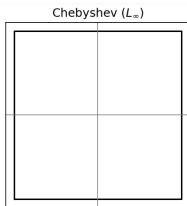
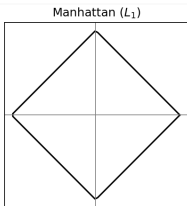
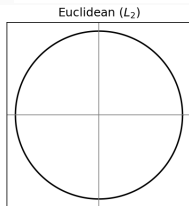
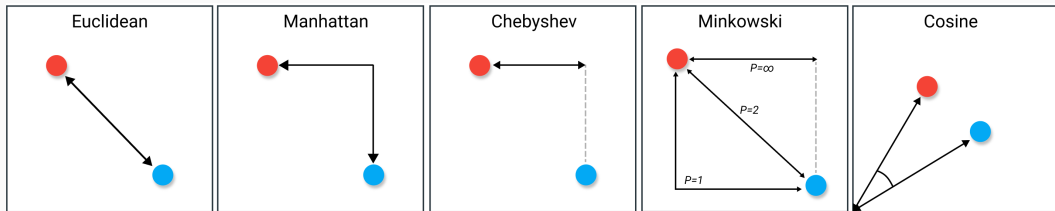
- Learns **representations** from raw unlabeled data by creating **pseudo-labels** from the data itself.
- Training task is often a **pretext task**, solved to enable downstream tasks.

Examples of pretext tasks

- Predicting missing parts of input (e.g., masked words in text, missing pixels in images).
- Contrastive learning: bring **similar samples closer**, push **dissimilar apart**.
- Next frame prediction in videos, **prediction of next word in sentence (LLMs)**.

Recapitulation of simplest algorithms (from BPC-UIN)

Distances and vector norms



Unit balls

$$\|x\|_2 = \sqrt{\sum_i x_i^2} \quad \|x\|_1 = \sum_i |x_i| \quad \|x\|_\infty = \max_i |x_i| \quad \|x\|_p = \left(\sum_i |x_i|^p \right)^{1/p}$$

For distance - let $x = (x_1, \dots, x_n) = (a - b)$ be the difference vector. [code example](#)

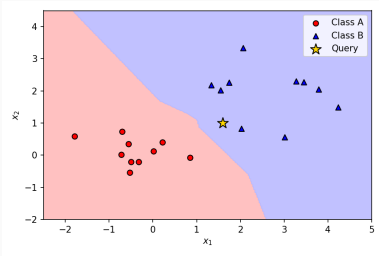
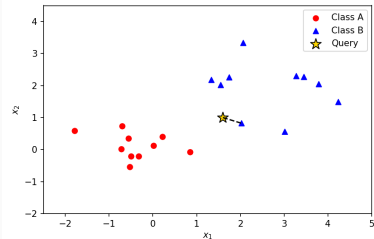
Nearest Neighbor (1-NN)

Idea

- For a new sample x_* , find the *closest* training point according to a chosen distance metric (typically Euclidean).
- Predict the same class/value: $\hat{y}(x_*) = y_{(1)}$

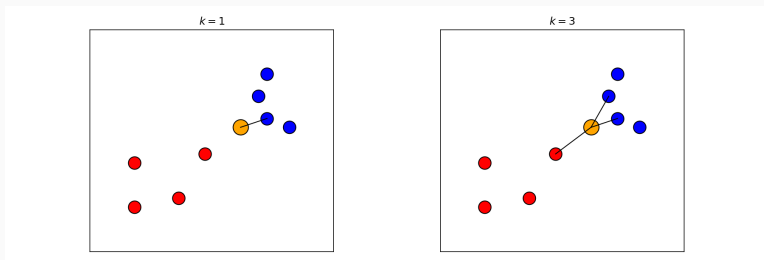
Properties

- No explicit training phase (lazy learning).
- Sensitive to noise, scaling of features, and outliers.
- Distance search is expensive for large n (use kd-tree).



k-Nearest Neighbors (k-NN)

- For x_* , find the k closest neighbors $\mathcal{N}_k(x_*)$.
- Majority vote - classification. Average - regression.
- Choice of k to regularize (bias-variance trade-off).
- Simple and powerful classifier.



playground: <http://vision.stanford.edu/teaching/cs231n-demos/knn/>
code example

k-means Clustering

Idea

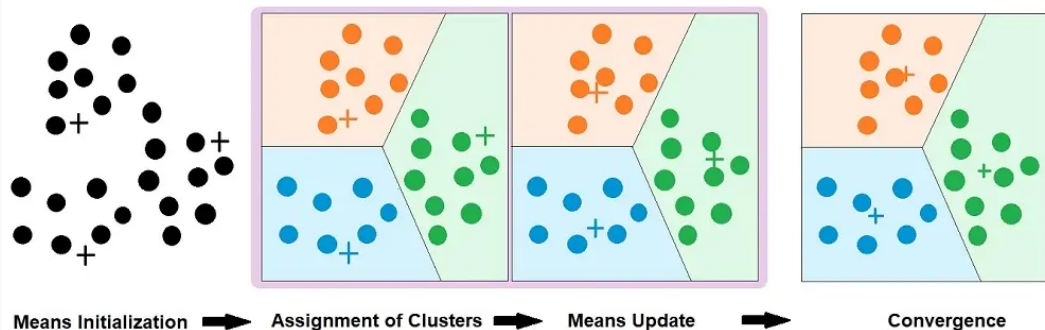
- Partition n data points into k clusters.
- Each cluster represented by its centroid (mean).
- Minimize the **inertia** (within-cluster variance):

$$J = \sum_{j=1}^k \sum_{x_i \in C_j} \|x_i - \mu_j\|_2^2$$

Algorithm (Lloyd's)

1. Initialize k centroids (random samples).
 2. Assign each x_i to nearest centroid.
 3. Update centroids as means of assigned points.
 4. Repeat 2–3 until convergence.
- Sensitive to initialization (use multiple restarts).
 - Finds local minima, not guaranteed global optimum.

k-means Clustering



code example

UPGMA (Unweighted Pair Group Method With Arithmetic Mean)

Idea

- A hierarchical clustering method.
- Iteratively merges the two closest clusters.
- Distance between clusters = average of all pairwise distances.
- Produces a rooted tree (dendrogram).

Algorithm

1. Start with each sample as its own cluster.
2. Find the pair of clusters with smallest distance.
3. Merge them into a new cluster.
4. Update the distance matrix using averages.
5. Repeat until one cluster remains.

Cluster distance definition

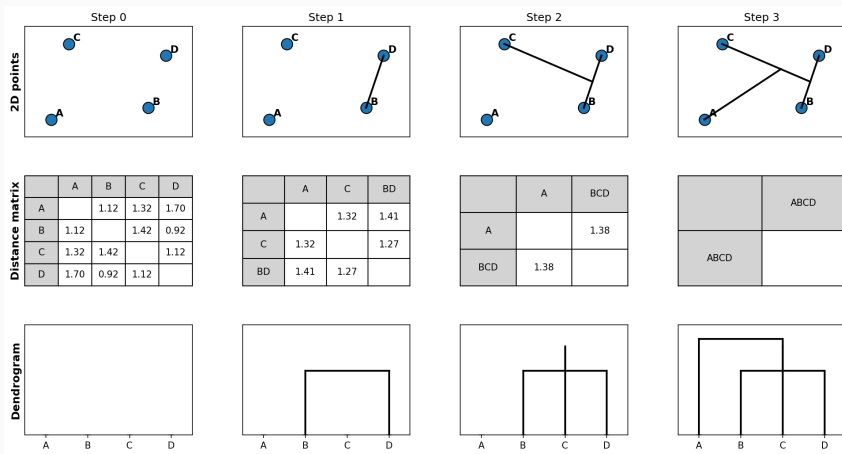
Let C_i and C_j be clusters to be merged.

$|C_i|$ is the number of elements in C_i .

The distance to another cluster C_k is updated as:

$$d(C_{i \cup j}, C_k) = \frac{|C_i| \cdot d(C_i, C_k) + |C_j| \cdot d(C_j, C_k)}{|C_i| + |C_j|}$$

UPGMA - practical construction



code example

Optimization

Mathematical Background for ML

Key areas of knowledge:

- **Linear Algebra:** vectors, matrices, matrix operations, eigenvalues/eigenvectors. – Matrices are an efficient way to represent complex calculations.
- **Calculus:** derivatives, gradients, partial derivatives, chain rule, [optimization concepts](#). – Optimization is key for model training, and it often uses derivatives.
- **Probability and Statistics:** random variables, distributions, expectation, variance, Bayes' theorem. – Provide tools to describe randomness, estimate patterns from data, and decide how likely events are.

Let's recapitulate key concepts of optimization (more details in MPA-EAL).

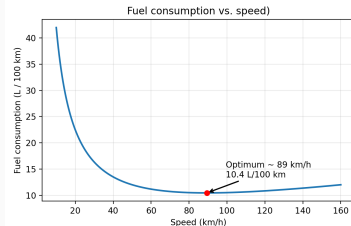
Optimization – intuition

- We are changing some inputs to get the best possible output.
- Process of finding the **best solution** among many possible ones.
- Typically means **minimizing a cost** or maximizing a reward.
- Imagine a **landscape**: optimization is searching for the lowest valley (minimum) or highest peak (maximum).
- Constraints define which parts of the landscape are **accessible**.
- Almost everything can be defined as an optimization problem!!
- Examples: investment portfolio selection - maximize profit; selecting the fastest route - minimizing time....
- Maximization can be done with the minimization of negative values.

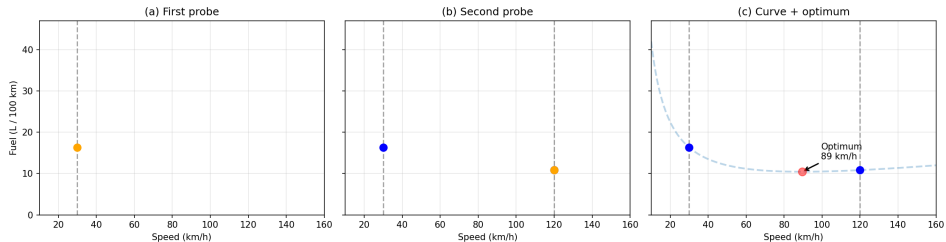
Optimization example 1D: speed vs. fuel consumption

Idea

At very low speed the ride is inefficient (idling losses, short gearing). At very high speed, aerodynamic drag $\propto v^2$ makes fuel consumption rise again. Therefore, an **optimal speed** naturally exists.

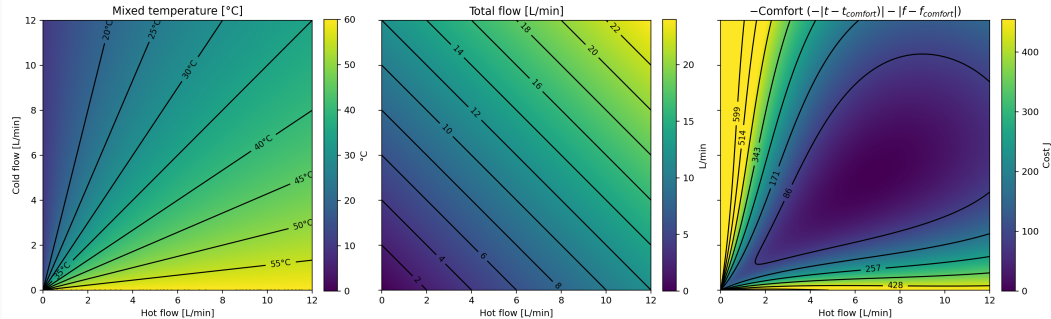


But the true curve is unknown — we can only probe it to search for the minimum:



Optimization example 2D: mixing hot and cold water

Mixing cold and hot water is an optimization task.
Need to set comfort flow and temperature.



Definition of optimization

Problem definition:

$$x^* = \arg \min_{x \in X} f(x)$$

Minimize the **objective function** $f(x)$ over the **domain** X to find the **minimizer** x^* .

Canonical form:

$$\begin{aligned} & \text{minimize} && f(x) \\ & \text{subject to} && g_i(x) \leq 0, \quad i = 1, \dots, m \\ & && h_j(x) = 0, \quad j = 1, \dots, p \end{aligned}$$

Where:

- $f(x)$ — objective function (cost, loss, or error) to minimize
- $g_i(x)$ — inequality constraint functions, restrict feasible region
- $h_j(x)$ — equality constraint functions
- $X = \{x \in \mathbb{R}^n : g_i(x) \leq 0, h_j(x) = 0\}$ — feasible set

Constrained optimization example: Mixing a Gin & Tonic

Task: Mix a Gin & Tonic from gin, tonic water, and lemon juice so the **glass is full**. Decision variables (ml): $\mathbf{x} = (x_{\text{gin}}, x_{\text{tonic}}, x_{\text{lemon}})^{\top} \geq 0$.

Objective (unknown, possibly non-linear): maximize taste score $T(\mathbf{x})$.

Simple constrained problem:

$$\begin{aligned} & \text{minimize} && -T(\mathbf{x}) \\ & \text{subject to} && \mathbf{1}^{\top} \mathbf{x} = V \quad (\text{glass is full}) \\ & && \mathbf{x} \geq 0 \end{aligned}$$

Convexity in optimization

Convex functions

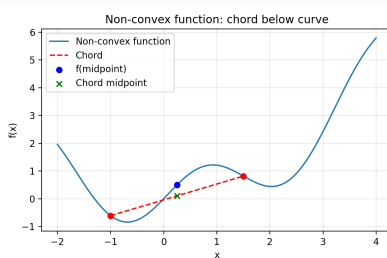
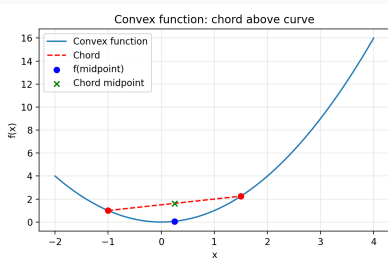
- A function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is convex if

$$f(\lambda x_1 + (1 - \lambda)x_2) \leq \lambda f(x_1) + (1 - \lambda)f(x_2)$$

for all x_1, x_2 and $\lambda \in [0, 1]$.

- Intuition: the function lies **below the chord** connecting any two points on it.
- One minimum and going downhill will reach minimum.

Easy



Hard

Convexity in optimization

Convex sets:

- A set $C \subseteq \mathbb{R}^n$ is convex if

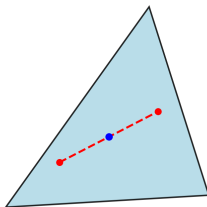
$$\lambda x_1 + (1 - \lambda)x_2 \in C$$

for all $x_1, x_2 \in C$ and $\lambda \in [0, 1]$.

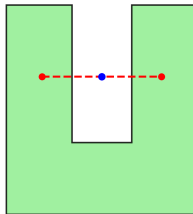
- Intuition: the line segment between any two points in C lies in C .
- Minimum is not behind barrier - I can go downhill to best point.

Easy

Convex set (midpoint inside)

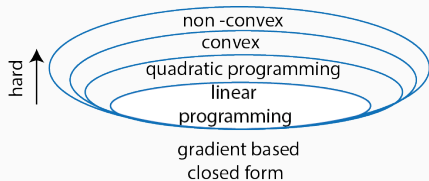


Non-convex set (midpoint outside)



Hard

Optimization approaches



- *Closed form (analytical solution)* - special case; we get an equation that provides a solution without iterations.
- *Gradient-based* - if we can calculate the derivative of $f(x)$, we can use it for faster optimization.
- *Linear programming* - linear $f(x)$ and constraints; can use standard fast solvers.
- *Quadratic programming* - quadratic $f(x)$ and linear constraints; can use standard fast solvers.
- *Convex* - typically easy to solve. One global minima.
- *Non-convex* - typically hard to solve. Typically, we use some heuristic without guarantees.
- *Discrete optimization* - usually NP-hard. Discrete domain.

Optimization approaches - examples

Almost everything (especially in ML) is optimization. We encounter it in many methods:

- *Closed form*: Ridge regression (with matrix inverse), Gaussian fitting, PCA (with eigenvectors).
- *Gradient-based*: Logistic regression, neural networks.
- *Linear programming*: L1 regression (least absolute deviations regression).
- *Quadratic programming*: Soft-margin SVM.
- *Convex*: Lasso regression, logistic regression.
- *Non-convex*: Deep neural networks, k-means clustering, GMM.
- *Discrete optimization*: Feature selection.

Closed form solution of speed vs. fuel consumption

Simple model:

$$\text{Fuel}(v) = \frac{A}{v} + Bv + C$$

Optimum condition (derivative equal to zero):

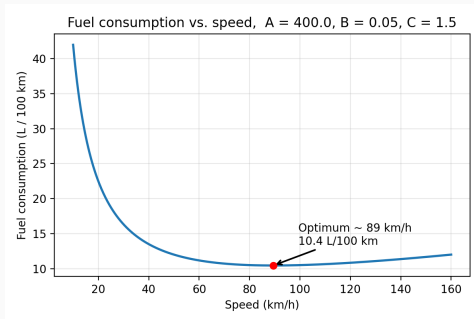
$$\frac{d}{dv}\text{Fuel}(v) = 0 \Rightarrow -\frac{A}{v^2} + B = 0$$

Solution:

$$v^* = \sqrt{\frac{A}{B}}$$

Since the second derivative is positive ($\frac{2A}{v^3} > 0$), the optimum is a **unique minimum**.

A – low-speed inefficiency coefficient, B – aerodynamic drag, C – baseline consumption (rolling resistance, losses)



Gradient descent

Intuition: Intuitively, if you change the input and recognize that the output (objective) is better, then you continue in this direction. If it is worse, then start changing in the opposite direction. (Let's think about it in terms of speed and water temperature.)

We want to move in the direction of the **steepest increase** = gradient direction.

The **gradient** $\nabla f(x)$ is a vector of partial derivatives:

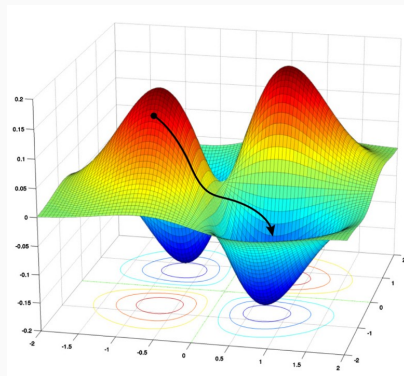
$$\nabla f(x) = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right).$$

Gradient descent update rule:

$$x^{(t+1)} = x^{(t)} - \eta \nabla f(x^{(t)})$$

where $\eta > 0$ is the **learning rate** (step size).

Very important for ML and artificial neural networks.



Recap Quiz

- **Classical programming vs. ML:** What are the inputs and outputs in each paradigm?
- **AI vs. ML vs. DL:** Which is the umbrella term and how do ML and DL differ?
- **Branches:** Give one task typical for *supervised*, *unsupervised*, *reinforcement*, and *self-supervised* learning.
- **Regression vs. classification:** What is predicted in each case?
- **Distances:** Write the formulas for $\|x\|_2$, $\|x\|_1$, and $\|x\|_\infty$ (for $x = a - b$).
- **1-NN vs. k-NN:** How does prediction differ and what is k trading off?
- **k-means:** What objective does it minimize and what makes it sensitive?

Recap Quiz

- **UPGMA:** How is the distance to a new merged cluster computed?
- **Optimization tasks:** Define 3 everyday tasks as an optimization problem.
- **Optimization tasks:** Define some image processing, signal processing, and bioinformatic tasks as optimization problems.
- **Convexity:** One property of convex functions/sets that simplifies optimization.
- **Gradient & GD:** Define $\nabla f(x)$ and write one GD update step.
- **GD:** Is gradient scalar or vector? How many dimensional?
- **Analytical optimization:** How to calculate the minimum of some function (e.g. $f(x) = 3x^2 + 5x + 1$) analytically?